

— EDITION 22

The Ledger

Crashing Problem of Sub-Penny Payments

An AI agent wants to buy 1,000 data tokens for \$0.002 each. If you route this over a traditional ledger, the database write costs more than the payment.

A 6-part breakdown

SWIPE >>>>

0101 / 06

The Database Write Scalability Wall.

Traditional relational databases and legacy banking core systems are built to process transactions sequentially. When thousands of machine agents trigger high-frequency micro-payments, database locking costs balloon.

0202 / 06

Why Traditional Relational Databases Explode.

The computing infrastructure costs required to log, audit, and clear a \$0.001 transaction through legacy clearing channels frequently exceed the actual economic value being transferred.

0303 / 06

The Concept of Streaming Money.

To handle machine-scale commerce, we must abandon discrete, batch-cleared payment events and transition to continuous, streaming value pipelines that clear fluidly alongside data utilization.

0404 / 06

| How State Channels and Layer-2 Rollups Solve It.

By routing high-frequency micro-payments off the main settlement layer using cryptographic state channels, platforms can aggregate thousands of sub-penny transactions into a single batch adjustment.

0505 / 06

The Implications for Content and Data Marketplaces.

Cracking high-throughput micro-payment architecture unlocks net-new business models, allowing platforms to price cloud compute, AI context tokens, and media streaming on a strict per-use basis.

0606 / 06

Save This: Making \$0.002 Payments Viable.

1) Discrete batch clearing can't price sub-penny events. 2) Stream value continuously alongside data usage. 3) Use state channels and L2 rollups to aggregate thousands of micro-payments into one settlement. 4) The unlock: *per-use pricing for compute, AI tokens, and media.*

— THE TAKEAWAY

What would you meter?

Comment what you'd price per-use if clearing were free. Repost to share it, and follow for daily agentic payments breakdowns.

◆ Joseph Bankole